FRONTEND · BIMESTRE 02

Revisão Geral

Aula 06. Sem código novo — amarrando Aulas 01 a 05: Nginx e arquivos estáticos, a árvore DOM e seus seletores, Vite e listeners, window/document, TypeScript e generics.

O Caminho até Aqui

- → **Aula 01** — arquivos estáticos e Nginx
- → **Aula 02** — árvore DOM, seletores, CSS
- → **Aula 03** — Vite, build, listeners, forms
- → **Aula 04** — window, document, cookies, JWT, CORS
- → **Aula 05** — TypeScript, generics, pré-compilação

Dois eixos crescendo juntos: a estrutura do frontend e a comunicação com o backend.

Por que Nginx em Docker

- 😔 Abrir o arquivo direto — não simula produção
- 😐 Backend só para servir estáticos — caro, desnecessário
- 😀 Nginx em Docker — ambiente de servidor real, sem custo de backend

---

`./public` local vira `/usr/share/nginx/html` no container.

try_files: a Cascata de Tentativas

nginx.conf

```
try_files $uri $uri/ =404;
1. $uri     → tenta como arquivo
2. $uri/    → tenta como diretório (+ index)
3. =404     → desiste, responde 404
```

Testa em ordem, usa o primeiro que existir — do mais específico ao mais genérico.

A Árvore DOM: Pais, Filhos, Irmãos

```
<body> pai de header, main, footer
├─ <header> filho de body; irmão de main
├─ <main>
│  └─ <article> descendente de body
└─ <footer> irmão de header e main
```

---

closest() sobe; children desce um nível — seguindo essa mesma árvore.

Os Cinco Métodos de Seleção

- `querySelector` — 1º elemento (seletor CSS) → **Element | null**
- `querySelectorAll` — todos (seletor CSS) → **NodeList**
- `getElementById` — por id → **Element | null**
- `getElementsByClassName` — por classe → **HTMLCollection**
- `getElementsByTagName` — por tag → **HTMLCollection**

O retorno reflete a cardinalidade do critério de busca.

id (1 para 1) vs class (n para n)

cardinalidade.js

```
// id: único no documento
document.getElementById("x") // Element | null
// class: n elementos podem compartilhar
document.getElementsByClassName("x") // HTMLCollection
```

Por isso busca por id nunca devolve lista — e busca por classe sempre pode.

Seletores CSS = Seletores do JS

seletores.css

```
A > B  /* filho direto */
A + B  /* irmão imediato */
[type="text"]  /* seletor de atributo */
:hover  /* pseudo-classe */
→ document.querySelectorAll('[type="text"]')
```

A mesma sintaxe estiliza no CSS e seleciona no JS.

Build e Vite: HMR

Build: transforma código-fonte no que vai para produção. O Vite é uma build tool.

```
Sem HMR: salva → recarrega página inteira → perde estado

Com HMR: salva → Vite identifica o módulo → só ele é substituído
```

target vs currentTarget

event.js

```
container.addEventListener("click", function (event) {
  event.target;         // onde ocorreu (pode ser filho)
  event.currentTarget; // onde o listener foi registrado
  event.target.closest(".item");
});
```

---

closest() acha o elemento clicável mesmo quando o clique bate num filho.

preventDefault() vs stopPropagation()

- `preventDefault()` — cancela o comportamento padrão (link navegar, form recarregar)
- `stopPropagation()` — impede a propagação pela árvore (bubbling)

Um cancela ação padrão; o outro cancela propagação. Não são a mesma coisa.

Forms e o Evento submit

form.js

```
formulario.addEventListener("submit", function (event) {
  event.preventDefault(); // evita reload
  const nome = formulario.nome.value;
});
```

Listener sempre no <form>, nunca só no botão de envio.

window acima de document

console.js

```
→ window.document === document
true
// document é a árvore DOM; window é o ambiente inteiro
window.localStorage, window.fetch, window.location
```

---

document é propriedade de window — não um objeto à parte.

DevTools, Cookies, JWT, CORS

- **DevTools** — Elements (DOM), Console (JS ao vivo), Network (cascata)
- **Cookies** — Max-Age, HttpOnly, Secure
- **JWT** — header.payload.signature; secret só no backend
- **CORS** — Access-Control-Allow-Origin libera outra origem

TypeScript e Generics

generics.ts

```
// navegador só entende JS puro — TS precisa ser transpilado
function wrap<T>(v: T): T { return v; }
→ clientApi.get<Usuario[]>("/users")
```

Uma função, infinitos tipos — Vite pré-compila resources/ → public/.

Resumo da Revisão

- **01** — Nginx + try_files (arquivo → diretório → 404)
- **02** — árvore DOM, 5 seletores, cardinalidade id/class
- **03** — build, HMR, target/currentTarget, preventDefault/stopPropagation
- **04** — document é propriedade de window; DevTools, cookies, JWT, CORS
- **05** — TypeScript, generics <T>, pré-compilação

TF Aula 06 — Revisão Geral

7 questões dissertativas, conceituais, sem escrita de código — uma para cada ponto enfatizado hoje.

https://docs.google.com/forms/d/e/1FAIpQLSdX_KVPNOB7EXGimv9pF8N2Q99ft_7oulpYz5XcNDYDZS7JJw/viewform?usp=publish-editor

---

Detalhes completos em TF06.md.